

Advanced JavaScript Array Practice Questions

A comprehensive collection of advanced array problems designed to challenge experienced developers and prepare for senior-level technical interviews. Each section includes a clear problem statement and a production-quality JavaScript solution.

1. Matrix Spiral Traversal

Problem: Given a 2D matrix, traverse it in spiral order and return elements as a flattened array.

JavaScript:

```
function spiralTraverse(matrix) {
  if (!matrix.length || !matrix[0].length) return [];
  const result = [];
  let top = 0, bottom = matrix.length - 1;
  let left = 0, right = matrix[0].length - 1;

  while (top <= bottom && left <= right) {
    // Traverse right
    for (let i = left; i <= right; i++) result.push(matrix[top][i]);
    top++;

    // Traverse down
    for (let i = top; i <= bottom; i++) result.push(matrix[i][right]);
    right--;

    // Traverse left
    if (top <= bottom) {
      for (let i = right; i >= left; i--) result.push(matrix[bottom][i]);
      bottom--;
    }

    // Traverse up
    if (left <= right) {
      for (let i = bottom; i >= top; i--) result.push(matrix[i][left]);
      left++;
    }
  }
  return result;
}
```

2. Custom Array Flattening with Depth Control

Problem: Implement deep array flattening with custom depth control and optional filtering predicate.

JavaScript:

```
function customFlatten(arr, depth = Infinity, predicate = () => true) {
  return arr.reduce((flat, item) => {
    if (Array.isArray(item) && depth > 0) {
      return flat.concat(customFlatten(item, depth - 1, predicate));
    }
    return predicate(item) ? flat.concat(item) : flat;
  }, []);
}

// Example
const nested = [1, [2, 3, [4, 5]], 6, [7, 8]];
const isEven = x => x % 2 === 0;
```

```
console.log(customFlatten(nested, 2, isEven)); // [2, 4, 6, 8]
```

3. Fenwick Tree (Binary Indexed Tree)

Problem: Implement a Fenwick Tree for efficient prefix and range sum queries with point updates.

JavaScript:

```
class FenwickTree {
  constructor(arr) {
    this.n = arr.length;
    this.tree = new Array(this.n + 1).fill(0);
    for (let i = 0; i < arr.length; i++) this.update(i, arr[i]);
  }
  update(index, delta) { // add 'delta' to position index
    index++; // 1-indexed
    while (index <= this.n) {
      this.tree[index] += delta;
      index += index & -index;
    }
  }
  prefixSum(index) {
    let sum = 0;
    index++; // 1-indexed
    while (index > 0) {
      sum += this.tree[index];
      index -= index & -index;
    }
    return sum;
  }
  rangeSum(left, right) {
    return this.prefixSum(right) - (left > 0 ? this.prefixSum(left - 1) : 0);
  }
}

// Usage
const ft = new FenwickTree([1,3,5,7,9,11]);
console.log(ft.rangeSum(1, 3)); // 3 + 5 + 7 = 15
```

4. Segment Tree for Range Operations

Problem: Implement a segment tree supporting range sum or min queries.

JavaScript:

```
class SegmentTree {
  constructor(arr, operation = 'sum') {
    this.n = arr.length;
    this.tree = new Array(4 * this.n);
    this.arr = arr;
    this.op = operation; // 'sum' or 'min'
    this.build(1, 0, this.n - 1);
  }
  build(node, start, end) {
    if (start === end) this.tree[node] = this.arr[start];
    else {
      const mid = (start + end) >> 1;
      this.build(node << 1, start, mid);
      this.build((node << 1) | 1, mid + 1, end);
      this.tree[node] = this.op === 'sum'
        ? this.tree[node << 1] + this.tree[(node << 1) | 1]

```

```

        : Math.min(this.tree[node << 1], this.tree[(node << 1) | 1]);
    }
}
query(node, start, end, left, right) {
    if (right < start || end < left) return this.op === 'sum' ? 0 : Infinity;
    if (left <= start && end <= right) return this.tree[node];
    const mid = (start + end) >> 1;
    const q1 = this.query(node << 1, start, mid, left, right);
    const q2 = this.query((node << 1) | 1, mid + 1, end, left, right);
    return this.op === 'sum' ? q1 + q2 : Math.min(q1, q2);
}
rangeQuery(left, right) {
    return this.query(1, 0, this.n - 1, left, right);
}
}
}

```

5. Advanced Multi-Dimensional Array Sorting

Problem: Sort by multiple criteria with custom comparators and per-field order.

JavaScript:

```

function multiSort(arr, sortBy) {
    return arr.sort((a, b) => {
        for (const { key, order = 'asc', compareFn } of sortBy) {
            const aVal = typeof key === 'function' ? key(a) : a[key];
            const bVal = typeof key === 'function' ? key(b) : b[key];
            const result = compareFn ? compareFn(aVal, bVal)
                : (aVal < bVal ? -1 : aVal > bVal ? 1 : 0);
            if (result !== 0) return order === 'desc' ? -result : result;
        }
        return 0;
    });
}

// Usage
const users = [
    { name: 'Alice', age: 30, salary: 50000 },
    { name: 'Bob', age: 25, salary: 60000 },
    { name: 'Charlie', age: 30, salary: 45000 }
];
const sorted = multiSort(users, [
    { key: 'age', order: 'desc' },
    { key: 'salary', order: 'asc' }
]);
console.log(sorted);

```

6. Longest Increasing Subsequence (O(n log n))

Problem: Find the length of the LIS using binary search on 'tails'.

JavaScript:

```

function longestIncreasingSubsequence(nums) {
    if (nums.length === 0) return 0;
    const tails = [];
    for (const num of nums) {
        let l = 0, r = tails.length;
        while (l < r) {
            const m = (l + r) >> 1;
            if (tails[m] < num) l = m + 1;
        }
        tails[l] = num;
    }
    return tails.length;
}

```

```

        else r = m;
    }
    if (l === tails.length) tails.push(num);
    else tails[l] = num;
}
return tails.length;
}

```

7. Maximum Rectangle in Binary Matrix

Problem: Find the area of the largest rectangle containing only 1s in a binary matrix.

JavaScript:

```

function maximalRectangle(matrix) {
    if (!matrix.length || !matrix[0].length) return 0;
    const cols = matrix[0].length;
    const heights = new Array(cols).fill(0);
    let maxArea = 0;

    for (let i = 0; i < matrix.length; i++) {
        for (let j = 0; j < cols; j++) {
            heights[j] = matrix[i][j] === '1' || matrix[i][j] === 1 ? heights[j] + 1 : 0;
        }
        maxArea = Math.max(maxArea, largestRectangleInHistogram(heights));
    }
    return maxArea;
}

function largestRectangleInHistogram(heights) {
    const stack = [];
    let maxArea = 0;
    for (let i = 0; i <= heights.length; i++) {
        const h = i === heights.length ? 0 : heights[i];
        while (stack.length && h < heights[stack[stack.length - 1]]) {
            const height = heights[stack.pop()];
            const width = stack.length === 0 ? i : i - stack[stack.length - 1] - 1;
            maxArea = Math.max(maxArea, height * width);
        }
        stack.push(i);
    }
    return maxArea;
}

```

8. Sliding Window Maximum (Deque)

Problem: Return maximums for all windows of size k using a deque for O(n) time.

JavaScript:

```

function slidingWindowMaximum(nums, k) {
    const dq = []; // store indices
    const res = [];
    for (let i = 0; i < nums.length; i++) {
        // drop indices outside window
        while (dq.length && dq[0] <= i - k) dq.shift();
        // drop smaller values from the back
        while (dq.length && nums[dq[dq.length - 1]] <= nums[i]) dq.pop();
        dq.push(i);
        if (i >= k - 1) res.push(nums[dq[0]]);
    }
}

```

```

    return res;
}

```

9. Count Inversions using Merge Sort

Problem: Count the number of index pairs (i, j) such that $i < j$ and $arr[i] > arr[j]$.

JavaScript:

```

function countInversions(arr) {
    function mergeSort(a) {
        if (a.length <= 1) return { arr: a.slice(), inv: 0 };
        const mid = a.length >> 1;
        const left = mergeSort(a.slice(0, mid));
        const right = mergeSort(a.slice(mid));
        const merged = [];
        let i = 0, j = 0, inv = left.inv + right.inv;
        while (i < left.arr.length && j < right.arr.length) {
            if (left.arr[i] <= right.arr[j]) merged.push(left.arr[i++]);
            else {
                merged.push(right.arr[j++]);
                inv += left.arr.length - i;
            }
        }
        while (i < left.arr.length) merged.push(left.arr[i++]);
        while (j < right.arr.length) merged.push(right.arr[j++]);
        return { arr: merged, inv };
    }
    return mergeSort(arr).inv;
}

```

10. Median of Two Sorted Arrays

Problem: Find the median of two sorted arrays in $O(\log(\min(m, n)))$ time.

JavaScript:

```

function findMedianSortedArrays(nums1, nums2) {
    if (nums1.length > nums2.length) return findMedianSortedArrays(nums2, nums1);
    const m = nums1.length, n = nums2.length;
    let low = 0, high = m;
    while (low <= high) {
        const i = (low + high) >> 1;
        const j = ((m + n + 1) >> 1) - i;
        const maxL1 = i === 0 ? -Infinity : nums1[i - 1];
        const minR1 = i === m ? Infinity : nums1[i];
        const maxL2 = j === 0 ? -Infinity : nums2[j - 1];
        const minR2 = j === n ? Infinity : nums2[j];
        if (maxL1 <= minR2 && maxL2 <= minR1) {
            if (((m + n) & 1) === 1) return Math.max(maxL1, maxL2);
            return (Math.max(maxL1, maxL2) + Math.min(minR1, minR2)) / 2;
        } else if (maxL1 > minR2) high = i - 1;
        else low = i + 1;
    }
    throw new Error("Input arrays are not sorted");
}

```

11. Async Iterator with Backpressure

Problem: Implement an async iterator that yields batches to avoid blocking the event loop.

JavaScript:

```
class AsyncArrayProcessor {
  constructor(array, batchSize = 100, delay = 10) {
    this.array = array;
    this.batchSize = batchSize;
    this.delay = delay;
    this.index = 0;
  }
  [Symbol.asyncIterator]() {
    return {
      next: async () => {
        if (this.index >= this.array.length) return { done: true };
        const batch = this.array.slice(this.index, this.index + this.batchSize);
        this.index += this.batchSize;
        await new Promise(r => setTimeout(r, this.delay));
        return { value: batch, done: false };
      }
    };
  }
}

// Usage
async function processLargeArray(largeArray) {
  const processor = new AsyncArrayProcessor(largeArray, 1000, 50);
  for await (const batch of processor) {
    console.log(`Processing batch of ${batch.length} items`);
  }
}
```

12. Advanced Trie for Auto-Completion

Problem: Implement a trie for efficient prefix matching with top suggestions by score.

JavaScript:

```
class TrieNode {
  constructor() {
    this.children = {};
    this.isEndWord = false;
    this.suggestions = []; // { word, score }
  }
}

class AutoCompleteTrie {
  constructor(maxSuggestions = 5) {
    this.root = new TrieNode();
    this.maxSuggestions = maxSuggestions;
  }
  insert(word, score = 1) {
    let node = this.root;
    for (const ch of word) {
      if (!node.children[ch]) node.children[ch] = new TrieNode();
      node = node.children[ch];
      this._updateSuggestions(node, word, score);
    }
    node.isEndWord = true;
  }
  _updateSuggestions(node, word, score) {
    node.suggestions.push({ word, score });
    node.suggestions.sort((a, b) => b.score - a.score);
    if (node.suggestions.length > this.maxSuggestions) {
      node.suggestions.length = this.maxSuggestions;
    }
  }
}
```

```

    }
  }
  searchPrefix(prefix) {
    let node = this.root;
    for (const ch of prefix) {
      if (!node.children[ch]) return [];
      node = node.children[ch];
    }
    return node.suggestions.map(s => s.word);
  }
}

```

13. Matrix Chain Multiplication (DP)

Problem: Find the minimum number of scalar multiplications needed for a chain of matrices.

JavaScript:

```

function matrixChainOrder(dim) {
  const n = dim.length - 1;
  const dp = Array.from({ length: n }, () => Array(n).fill(0));
  for (let L = 2; L <= n; L++) {
    for (let i = 0; i <= n - L; i++) {
      const j = i + L - 1;
      dp[i][j] = Infinity;
      for (let k = i; k < j; k++) {
        const cost = dp[i][k] + dp[k + 1][j] + dim[i] * dim[k + 1] * dim[j + 1];
        if (cost < dp[i][j]) dp[i][j] = cost;
      }
    }
  }
  return dp[0][n - 1];
}
// Example: dim = [1,2,3,4] => A1(1x2), A2(2x3), A3(3x4)
console.log(matrixChainOrder([1,2,3,4]));

```

14. Advanced Array Difference (Myers' Algorithm)

Problem: Compute an edit script between two arrays using Myers' O(ND) algorithm.

JavaScript:

```

function myersDiff(oldArr, newArr) {
  const N = oldArr.length, M = newArr.length, MAX = N + M;
  const v = { 1: 0 };
  const trace = [];
  for (let d = 0; d <= MAX; d++) {
    trace.push({ ...v });
    for (let k = -d; k <= d; k += 2) {
      const vkMinus = v[k - 1] ?? -Infinity;
      const vkPlus = v[k + 1] ?? -Infinity;
      let x;
      if (k === -d || (k !== d && vkMinus < vkPlus)) x = vkPlus;
      else x = vkMinus + 1;
      let y = x - k;
      while (x < N && y < M && oldArr[x] === newArr[y]) { x++; y++; }
      v[k] = x;
      if (x >= N && y >= M) return constructDiff(trace, oldArr, newArr, d);
    }
  }
  return [];
}

```

```

}
function constructDiff(trace, A, B, d) {
  let x = A.length, y = B.length;
  const out = [];
  for (let depth = d; depth >= 0; depth--) {
    const v = trace[depth];
    const k = x - y;
    let prevK;
    if (k === -depth || (k !== depth && (v[k - 1] ?? -Infinity) < (v[k + 1] ?? -Infinity))) {
      prevK = k + 1;
    } else {
      prevK = k - 1;
    }
    const prevX = v[prevK] ?? 0;
    const prevY = prevX - prevK;
    while (x > prevX && y > prevY) {
      out.unshift({ type: 'equal', value: A[x - 1] });
      x--; y--;
    }
    if (depth > 0) {
      if (x > prevX) { out.unshift({ type: 'delete', value: A[x - 1] }); x--; }
      else { out.unshift({ type: 'insert', value: B[y - 1] }); y--; }
    }
  }
  return out;
}

```

15. Persistent Data Structure — Immutable Array

Problem: Keep prior versions cheaply by recording modifications; each mutation creates a new version.

JavaScript:

```

class PersistentArray {
  constructor(data = [], version = 0, mods = new Map()) {
    this.data = data;
    this.version = version;
    this.modifications = mods; // Map(index -> value)
  }
  get(index) {
    return this.modifications.has(index) ? this.modifications.get(index) : this.data[index];
  }
  set(index, value) {
    const mods = new Map(this.modifications);
    mods.set(index, value);
    return new PersistentArray(this.data, this.version + 1, mods);
  }
  push(value) {
    const arr = this.toArray().slice();
    arr.push(value);
    return new PersistentArray(arr, this.version + 1, new Map());
  }
  toArray() {
    const out = this.data.slice();
    for (const [i, v] of this.modifications.entries()) out[i] = v;
    return out;
  }
  getVersion() { return this.version; }
}

// Usage
let a1 = new PersistentArray([1,2,3]);
let a2 = a1.set(1, 'modified');
let a3 = a2.push(4);
console.log(a1.toArray()); // [1,2,3]

```



```
console.log(a2.toArray()); // [1, 'modified', 3]  
console.log(a3.toArray()); // [1, 'modified', 3, 4]
```

Notes

These advanced problems cover complex algorithms, data structures, and real-world scenarios that senior developers encounter. They test deep understanding of algorithmic concepts, optimization techniques, and advanced JavaScript features.